

Connect — Campus Social Network

AI1220 Assignment 1: Requirements Engineering, Architecture & Proof of Concept

Member	Role	Owns
Jiadong Zhang	A — Backend core	Auth, communities, content, Audience Policy, data model
Nura	B — Real-time & CI	WebSocket gateway, collaboration, Redis, CI and infrastructure
Fatima	C — Frontend	React app, rich-text editor, accessibility
Zayed	D — Moderation, AI, quality	Moderation and appeals, AI triage, test strategy, documentation

Repository: <https://github.com/dereksodo/AmazingNetwork> · October 2026 · All data in this report and the proof of concept is fictional.

Contents

0 Product overview

- 0.1 Problem
- 0.2 Users
- 0.3 Scope
- 0.4 Content types
- 0.5 Interaction rules
- 0.6 Moderation policies

1 Requirements Engineering

- 1.1 Stakeholders
- 1.2 Functional requirements
- 1.3 Non-functional requirements
- 1.4 User stories and scenarios
- 1.5 Validation and traceability

2 System Architecture

- 2.1 Architectural drivers (ranked)
- 2.2 System design (C4)
- 2.3 Behaviour and design decisions
- 2.4 Interfaces
- 2.5 Code and repository structure
- 2.6 Data model
- 2.7 Architecture decision records

3 Project Management and Team Collaboration

- 3.1 Team structure
- 3.2 Development workflow
- 3.3 Development approach
- 3.4 Risks
- 3.5 Timeline for Assignment 2

4 Proof of Concept

0 Product overview

0.1 Problem

Campus life at our university is coordinated through scattered WhatsApp groups, mailing lists and Instagram accounts. Club announcements get lost, nobody can tell whether an account really belongs to a student, officers draft event posts in separate documents and paste them around, and when someone is harassed there is no accountable way to report it. **Connect** gives the university one verified place where students and student groups share updates, discuss topics, co-write announcements and events, message each other, and decide who sees what, with transparent moderation.

0.2 Users

User	Description
Student	Verified university member (staff accounts have the same capabilities). The default user.
Group officer	A student with extra rights <i>inside one community</i> (officer or owner role in a membership).
Moderator	Appointed by the school admin. Handles reports for the whole campus.
School admin	University staff. Approves communities, appoints moderators, posts campus-wide announcements, reads the audit log.

"Non-member" is not an account type: it describes a student's relationship to a community they have not joined. The development team is a stakeholder (§1.1) but has **no in-app role** and no access to user content.

0.3 Scope

First release (specified and designed here)	Later	Out of scope
University SSO sign-in (students and staff); profiles; communities with approval; posts, discussions, announcements, events with RSVP; comments and reactions; three visibility levels; following and blocking; real-time co-editing of announcement/event drafts; live notifications; 1:1 direct messages; reports, moderation actions, appeals, audit log; AI report triage; web app responsive on phones	Shorts (short videos); polls; group chats; native mobile apps and push notifications; calendar export; search ranking and recommendations; Arabic UI	Access for people outside the university (incl. alumni, pending Q1); anonymous posting; marketplace or payments; live video; integration with the learning management system or grades

0.4 Content types

Type	Created by	Lives in	Format and limits
Post	Students, officers, moderators	Own profile or a joined community	Rich text (allowlisted HTML), ≤ 5,000 chars, ≤ 4 images of ≤ 5 MB
Discussion	Same as post	A joined community	Post + required title; can be locked
Announcement	Officers (community); school admin (campus-wide)	Community or campus	Written as a collaborative draft; notifies members; ≤ 3 pinned per community
Event	Officers; school admin	Community or campus	Announcement + start/end, location, capacity, RSVP deadline
Draft	Its creator; officers share community drafts	Not published	Rich text edited live; visible only to collaborators
Comment	Anyone who can see the parent	Under any published item	Plain text + links, ≤ 1,000 chars, one level of replies
Reaction	Anyone who can see the item	On any published item	One of 👍❤️🎉👎, one of each per user
Direct message	Students, officers	1:1 conversation	Plain text, ≤ 2,000 chars

0.5 Interaction rules

Who can create what

	Student	Officer	School admin	Non-member of the community	Moderator
Post / discussion	✓	✓	✗	✗	✓
Announcement / event	✗	✓	✓ campus-wide	✗	✗
Comment / reaction	✓	✓	✓	✗	✓
Direct message	✓	✓	✗	✓ unless blocked	✗ uses system notices
Draft	own	own + shared community drafts	own	–	–

Rules

1. *View before interact.* Only people who can see an item can comment on, react to, RSVP to, reshare or report it.
2. *Comments inherit visibility* from their parent; changing a post's visibility changes theirs.
3. *Audiences never widen.* Only campus-visible posts can be reshared.

4. *Mentions do not leak.* Mentioning someone outside the audience creates no notification.
5. *Editing.* Posts are editable any time (marked "edited", versions kept); comments for 15 minutes.
6. *Author and officer controls.* Authors can disable comments; officers can lock discussions and pin up to 3 items.
7. *Blocking is mutual invisibility:* no viewing, mentioning, commenting, following or messaging.
8. *Following.* Users may require follower approval. Following a community does not grant access to community-only posts; membership does.
9. *Events.* RSVPs close at the deadline; when capacity is reached new RSVPs are waitlisted and promoted in order when someone cancels; changes notify everyone who RSVPed.
10. *Notifications* for replies, mentions, announcements in joined communities and event changes; communities can be muted.
11. *Hidden content is frozen:* no new comments, reactions or reshares while hidden.

0.6 Moderation policies

#	Policy
P1	Moderators can hide content (reversible, shown to the author with the reason) or remove it (purged after 30 days). Every action needs a reason category.
P2	Moderators contact users through system notices from the "Connect Moderation" account. Users cannot reply; they can appeal.
P3	Direct messages are private. Moderators never browse conversations. When a user reports a DM, only the messages the reporter selects are copied into the report.
P4	Timed mute for 24 h, 7 days or 30 days: the user can read but cannot post, comment, react or message.
P5	Appeals: one per action, within 7 days , decided by a <i>different</i> moderator.
P6	Every moderation and admin action is written to an append-only audit log .
P7	Officers can hide comments inside their own community and escalate to campus moderators.
P8	The AI only suggests ; a human takes every moderation action.

1 Requirements Engineering

1.1 Stakeholders

ID	Stakeholder	Goals	Concerns	Influence
SH-1	Students	Find events and groups, discuss topics, message friends	Privacy, noise, harassment, being silenced unfairly	High — adoption decides success
SH-2	Group officers	Reach members, co-write announcements without version chaos, fill events	Reach, editing conflicts, low turnout	Medium — produce most valuable content
SH-3	Moderators	Deal with harmful content quickly and fairly	Workload, unclear policy, being blamed, exposure to disturbing content	Medium — define day-to-day safety
SH-4	School admin (student affairs, IT, legal)	Safe, lawful, accountable platform; only real university members	Data protection, liability, reputation, audit	High — can approve or shut down the product
SH-5	Dev team (us)	Deliver a working, defensible system in the time given using FastAPI, React, WebSockets and an LLM	Scope, complexity, unfamiliar tech	High — decides what is feasible

Stakeholder needs (used in the traceability matrix)

ID	Need	From
SN-1	Share updates and discuss topics with the campus or a group	SH-1, SH-2
SN-2	Control who sees my content	SH-1, SH-2
SN-3	Private 1:1 conversations, with protection from harassment	SH-1
SN-4	Co-write announcements and events and reach members	SH-2
SN-5	Handle reports quickly and fairly with limited moderator time	SH-3
SN-6	Verified members only; accountability and auditability	SH-4
SN-7	Reach the whole campus with official announcements	SH-4
SN-8	Buildable and testable with the required technologies	SH-5
SN-9	Keep working as participation grows	SH-1 – SH-4

Conflicts and how we resolved them

Conflict	Resolution in the requirements
Students want DMs private (SH-1) vs moderators/admin need to act on harassment in DMs (SH-3, SH-4)	P3: moderators see only the messages a reporter chooses to attach (FR-MOD-8).
Officers want maximum reach (SH-2) vs students want less noise (SH-1)	Only officers announce; max 3 pins; users can mute a community (FR-INT-5).
Admin wants long audit retention (SH-4) vs students' privacy and deletion (SH-1)	Audit log keeps 1 year of <i>metadata</i> (who, what, when, why) but not content; content is purged 30 days after removal or account deletion (NFR-7, NFR-10).
Moderators want AI help (SH-3) vs students' content going to an external LLM (SH-1, SH-4)	Advisory only, no identifiers sent, mock provider by default, provider switchable (FR-AI-6, ADR-004); external processing is open question Q3.
Fast removal of harmful content (SH-4) vs fairness and free expression (SH-1)	Hide (reversible) before remove; 7-day appeal to a different moderator (FR-MOD-3, FR-MOD-5).
Feature wishes (shorts, mobile app) vs team capacity (SH-5)	Shorts and native apps moved to "later"; responsive web only.

1.2 Functional requirements

Each requirement describes observable behaviour and can be tested. "Shall" marks required behaviour; *design choices* that implement it are recorded separately (table at the end of this section and the ADRs).

Identity (FR-ID)

ID	Requirement
FR-ID-1	The system shall allow sign-in only through university SSO; accounts whose email is not on a university domain shall be refused with the message "Connect is only for university members".
FR-ID-2	On first sign-in the system shall create a profile with display name and programme, editable by the user.
FR-ID-3	The system shall support the global roles user, moderator and admin, and the per-community roles member, officer and owner. Only an admin can grant the moderator role.
FR-ID-4	A session shall expire after 7 days of inactivity; signing out shall invalidate it immediately on all open tabs.
FR-ID-5	A user shall be able to request account deletion; the account becomes invisible immediately and its data is purged within 30 days.

Communities (FR-COM)

ID	Requirement
FR-COM-1	Any user shall be able to request a new community (name, description, open/closed); it becomes visible only after an admin approves it. The requester becomes owner.
FR-COM-2	Joining an <i>open</i> community shall take effect immediately; joining a <i>closed</i> community shall create a request that an officer accepts or rejects.
FR-COM-3	Owners shall be able to promote members to officer, demote officers and remove members.
FR-COM-4	A user who leaves or is removed shall lose access to community-only content on their next request and stop receiving its live events.

Publishing (FR-PUB)

ID	Requirement
FR-PUB-1	Users shall be able to publish posts with rich text (bold, italic, underline, headings, lists, links, quotes, code) and up to 4 images, on their profile or in a joined community.
FR-PUB-2	Members shall be able to start discussions (title required) in a community; officers can lock a discussion so it stays readable but accepts no new comments.
FR-PUB-3	Officers shall be able to publish announcements in their community; school admins campus-wide. Publishing notifies the audience and may pin the item (≤ 3 pinned per community).
FR-PUB-4	Events shall have start/end time, location, capacity and RSVP deadline. Members RSVP <i>going</i> or <i>interested</i> ; beyond capacity they are <i>waitlisted</i> and promoted in order when a place frees up. Editing or cancelling an event notifies everyone who RSVPed.
FR-PUB-5	Authors shall be able to edit and delete their content. Edited posts show "edited" and keep previous versions; deleting a post hides its comments.
FR-PUB-6	The server shall remove any HTML tag or attribute that is not on the allowlist before storing content.

Real-time collaboration (FR-COL)

ID	Requirement
FR-COL-1	An officer shall be able to create an announcement or event draft in their community and invite other officers of that community as collaborators.
FR-COL-2	Collaborators editing the same draft shall see each other's changes and cursors/presence while editing.
FR-COL-3	Simultaneous edits, including to the same sentence, shall be merged so that no collaborator's text is lost and all collaborators end with identical content.
FR-COL-4	If a collaborator loses connection, the editor shall show "offline", keep their edits, and merge them when the connection returns.
FR-COL-5	Publishing a draft shall be an explicit action by an officer. If two officers publish at the same time, exactly one item is published and the other officer is told it was already published. After publishing, the draft becomes read-only.
FR-COL-6	A draft shall be visible only to its collaborators.

Audience control (FR-AUD)

ID	Requirement
FR-AUD-1	Every post shall have one visibility: campus (all signed-in users), community (members of its community) or followers (the author's followers; profile posts only).
FR-AUD-2	The system shall never deliver an item to a user outside its audience through any path: single fetch, feed, search, comments, notifications or live events. A request for such an item returns the same "not found" response as a non-existent item.
FR-AUD-3	A user shall be able to block another user; neither can then see the other's content, mention, comment on, follow or message the other.
FR-AUD-4	Changing a post's visibility shall apply to the post and its comments from the next request and the next live event.
FR-AUD-5	Only campus-visible posts can be reshared.
FR-AUD-6	Mentioning a user who cannot see the item shall not notify them.

Interaction (FR-INT)

ID	Requirement
FR-INT-1	Users shall be able to comment ($\leq 1,000$ chars, one reply level) on items they can see, unless comments are disabled or the thread is locked. Comments can be edited for 15 minutes.
FR-INT-2	Users shall be able to add or remove one reaction of each kind on items they can see.
FR-INT-3	Users shall be able to follow users and communities; a user who requires approval accepts or rejects follow requests.
FR-INT-4	Each user shall have a feed of visible items from followed users and joined communities, newest first, that updates live.
FR-INT-5	Users shall receive live notifications for replies, mentions, new announcements in joined communities, event changes and system notices, and can mute a community.
FR-INT-6	Users shall be able to exchange 1:1 direct messages, delivered live; only the two participants can read a conversation. Blocked or muted users cannot send messages.

Moderation (FR-MOD)

ID	Requirement
FR-MOD-1	Users shall be able to report any post, comment, user or direct message they can see (not their own), choosing a reason; each user can report an item once. The reporter sees a confirmation.
FR-MOD-2	Moderators shall see a queue of open reports, ordered by AI severity (high first) and then age, with report count and AI suggestion.
FR-MOD-3	Moderators shall be able to dismiss a report, hide or remove content, send a system notice, or mute the user for 24 h, 7 days or 30 days; each action requires a reason.
FR-MOD-4	The affected author shall be notified of any action with its reason; hidden content stays visible to its author with a banner and accepts no new interactions.
FR-MOD-5	The affected user shall be able to appeal an action once within 7 days. The appeal is assigned to a different moderator, whose decision (upheld or overturned) is final and notified to the user. Overturning restores the content or lifts the mute.
FR-MOD-6	Every moderation and admin action shall be recorded in an audit log that cannot be edited or deleted through the application. Admins can export it for a date range.
FR-MOD-7	Officers shall be able to hide comments in their own community and escalate them to campus moderators.
FR-MOD-8	Moderators shall not be able to open direct-message conversations. A DM report shall contain only copies of the messages the reporter selected.
FR-MOD-9	System notices shall be sent from the "Connect Moderation" account and shall not accept replies.

AI feature: Report Triage Assistant (FR-AI)

Whose need: moderators (SN-5), whose workload grows with participation; indirectly students, because severe reports are seen first. *Data used:* the reported text (or the attached DM excerpts), the report reason and the list of policy categories; never names, user IDs, profile data or unreported content. *How output is checked:* schema and enum validation, mandatory human decision, recorded moderator agreement, and an offline evaluation set (NFR-12).

ID	Requirement
FR-AI-1	When a report is filed, the system shall request a triage suggestion containing a category (from the policy list), a severity (low / medium / high) and a rationale of at most 300 characters.
FR-AI-2	The suggestion shall be shown in the queue labelled "AI suggestion", and used to order the queue.
FR-AI-3	The AI shall never hide, remove, notify or mute; only a moderator can act.
FR-AI-4	If the LLM times out, fails, or returns output that is not valid, the report shall still appear in the queue, marked "unscored", within 5 seconds of being filed.
FR-AI-5	When deciding, the moderator shall record whether they agree with the suggestion; agreement rates are visible to admins.
FR-AI-6	Text sent to the LLM shall contain no user names or identifiers.

Technology constraints

ID	Constraint (given by the course)
TC-1	Backend in Python with FastAPI.
TC-2	Live updates and collaboration over WebSockets.
TC-3	React frontend with rich-text editing.
TC-4	Integration with a large language model (we chose DeepSeek behind an adapter; mock allowed).
TC-5	Fictional data only in the proof of concept; external systems may be mocked.

Required behaviour vs design choice

Required behaviour (what)	Design choice (how)	Recorded in
Collaborators' edits merge without loss (FR-COL-3)	Yjs CRDT with TipTap	ADR-002
Items never reach people outside the audience (FR-AUD-2)	Central Audience Policy, 404 for hidden items	ADR-003
Reports reach the queue even when the AI fails (FR-AI-4)	Asynchronous worker job, mock provider	ADR-004
Live notifications and feed updates (FR-INT-4/5)	WebSocket channels + Redis pub/sub	§2.3
Audit log cannot be altered (FR-MOD-6)	Insert-only table, DB role without UPDATE/DELETE	§2.6

1.3 Non-functional requirements

ID	Quality	Target	How measured	Why this target
NFR-1	Performance	p95 ≤ 300 ms for REST reads and ≤ 500 ms for writes at 500 concurrent users	k6 load test against staging	Feels instant; 500 ≈ evening peak for a 15k-student campus
NFR-2	Live latency	p95 ≤ 1 s for live events (posts, comments, DMs); ≤ 500 ms for co-editing updates between two editors	Timestamped test clients	Chat and co-editing feel broken above ~1 s

ID	Quality	Target	How measured	Why this target
NFR-3	Scalability	≥ 1,000 WebSocket connections per API instance; a second instance doubles capacity with no code change; data volume 15k users / 1M posts	Load test with 1 and 2 instances	Growth in participation (customer request); stateless API
NFR-4	Availability	99.5 % per month during semester; maintenance only 01:00–06:00	Uptime probe every minute	≈ 3.6 h/month downtime is tolerable for a non-critical campus tool, affordable for us
NFR-5	Recovery	RPO ≤ 15 min, RTO ≤ 1 h; ≤ 5 s of draft edits lost if the server crashes	Restore drill each sprint in staging; crash test	Posts and drafts are expensive to recreate
NFR-6	Security	TLS everywhere; authorization on every REST call and WebSocket message; HttpOnly SameSite cookies; HTML allowlist; rate limits: 10 posts/h, 60 comments/h, 30 DMs/min, new accounts (< 24 h) 3 posts/day and no links; 0 high findings in OWASP ZAP baseline scan	Automated tests, ZAP in CI, dependency audit	Prevent leaks, XSS, spam and account abuse
NFR-7	Privacy	Collect only profile data needed; deleted accounts and removed content purged within 30 days; staff cannot read DMs except reported excerpts; no identifiers sent to the LLM	Data inventory review; purge job test	Data-protection law and student trust
NFR-8	Usability	≥ 4 of 5 first-time test users join a community and publish a post within 3 minutes without help; SUS ≥ 70	Moderated usability test each milestone	Adoption depends on the first minutes
NFR-9	Accessibility	WCAG 2.2 AA; 0 serious/critical axe-core violations on main pages; editor fully keyboard-operable; all controls labelled for screen readers	axe in CI + manual keyboard test	Inclusion; university legal duty
NFR-10	Auditability	100 % of moderation and admin actions logged with actor, time, target and reason; insert-only; kept 1 year; 1-month export in ≤ 10 s	Test that every action endpoint writes a log row	Appeals, accountability (SN-6)
NFR-11	Maintainability / testability	CI on every PR; ≥ 80 % line coverage for Audience Policy and Moderation; a contract test for every endpoint	CI report	These modules carry the highest risk
NFR-12	AI quality	On a 100-item fictional labelled set: ≥ 80 % severity agreement and no "high" item rated "low"; p95 triage ≤ 15 s	Offline evaluation script per prompt version	Advice must be worth reading and never hide urgent cases

1.4 User stories and scenarios

US-1 — Officer co-writes an event (normal use, concurrency, failure). *As a Robotics Club officer, I want to write the open-day announcement together with the other officers at the same time, so that we publish one agreed version.*

- Given officers A and B have the draft open, when A types, then B sees the change within 500 ms and sees A's cursor.
- Given A and B edit the same sentence at the same moment, then both edits are kept and both screens show identical text.
- Given B loses Wi-Fi and keeps typing, then B sees "offline"; when B reconnects, B's edits appear for A and nothing is lost.
- Given both press Publish at once, then exactly one event is created; the other officer sees "Already published" and a link to it.
- Given a club member who is not an officer opens the draft link, then they get "not found".

US-2 — Student posts to their community only (audience policy). *As a student, I want to post "Robotics team needs a CAD helper" for Robotics Club members only, so that outsiders do not see it.*

- Given Aisha posts with visibility *community* in Robotics Club, then members see it in their feed within 1 s.
- Given Omar is not a member, when he opens the post's link, then he gets the same "not found" as for a non-existent post, and it never appears in his feed, search or notifications.
- Given Aisha mentions Omar in the post, then Omar receives no notification.
- Given a member leaves the club, then the post disappears from their next feed load and they get no more live events for it.

US-3 — Moderator triages reports, AI unavailable (failure, AI). *As a moderator, I want reports ordered by severity, so that I deal with the most harmful first.*

- Given a report is filed and the AI works, then it appears in the queue within 15 s with category, severity and rationale labelled "AI suggestion".
- Given the LLM provider is down, then the report appears within 5 s marked "unscored" and can be handled normally.
- Given the AI rates a post "high", then the post is **not** hidden until a moderator acts.
- When the moderator hides the post with reason "harassment" and marks "disagree with AI", then the author is notified, an audit entry is written and the agreement statistic is updated.

US-4 — Author appeals a hidden post (exception, policy). *As a student whose post was hidden, I want to appeal, so that a mistaken decision can be reversed.*

- Given my post was hidden 3 days ago, then I see it with a banner showing the reason and an "Appeal" button.
- When I submit an appeal, then it is assigned to a moderator other than the one who hid the post.
- When I try to appeal the same action again, then I see "You have already appealed this decision".
- Given 8 days have passed since the action, then the Appeal button is gone and the API answers "appeal window closed".

- When the appeal is overturned, then the post is visible again with its original visibility and I am notified.

US-5 — Student reports harassment in DMs (privacy policy). *As a student receiving abusive direct messages, I want to report them without exposing my whole conversation.*

- When I choose "Report" in a conversation, then I can select which messages to attach (at least one).
- Then the moderator sees only copies of the selected messages and the reason; they cannot open the conversation.
- Given the sender is muted for 7 days, then they cannot send me (or anyone) messages, and they receive a system notice they cannot reply to.
- Given I block the sender, then they can no longer message me or see my content.

US-6 — School admin publishes and audits (admin, auditability). *As a student-affairs admin, I want to post a campus-wide announcement and review moderation activity for the month.*

- When I publish a campus-wide announcement, then every signed-in user receives a notification and it is pinned at the top of the campus feed.
- When I export the audit log for September, then I receive a CSV within 10 s with actor, time, action, target and reason for every moderation action, and the export itself is logged.
- Given a moderator (not an admin) requests the export, then they are refused.

1.5 Validation and traceability

How we reviewed the requirements. (1) Each member reviewed another member's area against a checklist: observable? testable? one requirement per row? consistent with other areas? linked to a stakeholder need? (2) We walked through every user story step by step and checked that each step was covered by a requirement and an architecture component. (3) We replaced vague words ("fast", "secure", "easy") with numbers. (4) Every change was recorded in `DECISIONS.md`.

Problems found and resolved

Problem	Resolution
"Posts appear quickly" was not testable	NFR-2: $p95 \leq 1$ s
"Public" was ambiguous (internet or campus?)	Renamed <i>campus</i> : all signed-in university users
Early draft allowed only campus visibility, contradicting "users control who sees their content"	Three levels (FR-AUD-1)
"Moderators can see everything" contradicted DM privacy	Exception FR-MOD-8 and policy P3
"Delete posts" did not say whether deletion is reversible	Hide (reversible) vs remove (purged after 30 days)
Appeal window stated as 14 days in one place and 7 in another	Team decision: 7 days everywhere
"Non-member" and "dev team" listed as user types	Non-member is a relationship; dev team is a stakeholder only
Shorts would need video storage and moderation we cannot specify in time	Moved to "later"

Assumptions

ID	Assumption
A-1	The university offers SSO with OpenID Connect; until then it is mocked.
A-2	About 15,000 students and staff; peak 500–1,000 concurrent users.
A-3	English-only interface in the first release.
A-4	Moderators are trained staff and student volunteers appointed by the school admin.
A-5	Sending minimized report text to DeepSeek is acceptable for development; production use needs approval (Q3).
A-6	Staff accounts behave like student accounts unless given the admin role.
A-7	Assignment 2 lasts about six weeks.

Open questions

ID	Question	Owner
Q1	May alumni join?	School admin
Q2	What audit-log retention does university policy or law require (we assume 1 year)?	School admin / legal
Q3	May reported content be processed by an external LLM provider (data location, retention)? If not, we switch to a local model through the adapter.	School admin / IT
Q4	Who trains moderators, and is there a code of conduct we must link to?	School admin
Q5	Should staff (non-admin) be allowed to create communities?	School admin

Traceability matrix. Component IDs refer to §2.2 (C- *containers*, K- API components).

Need	Requirements	Stories	Architecture components	ADR
SN-1 share and discuss	FR-PUB-1/2/5/6, FR-INT-1/2/4	US-2	C-WEB, K-POST, K-POLICY, C-DB	ADR-003
SN-2 control audience	FR-AUD-1-6, FR-COM-4	US-2	K-POLICY, K-POST, K-RT	ADR-003
SN-3 private messages, safe from harassment	FR-INT-6, FR-AUD-3, FR-MOD-1/8	US-5	K-DM, K-MOD, K-RT	ADR-003
SN-4 co-write and reach members	FR-COL-1-6, FR-PUB-3/4	US-1	K-COLLAB, K-RT, K-BUS, C-REDIS	ADR-002
SN-5 handle reports quickly and fairly	FR-MOD-1-5, FR-AI-1-6	US-3, US-4	K-MOD, K-AI, C-WRK	ADR-004
SN-6 verified, accountable	FR-ID-1/3, FR-MOD-6, NFR-10	US-4, US-6	K-AUTH, K-AUDIT, K-MOD	ADR-001
SN-7 campus announcements	FR-PUB-3, FR-INT-5	US-6	K-POST, K-NOTIF	—
SN-8 buildable and testable	TC-1-5, NFR-11	all	repository structure, CI	ADR-001
SN-9 cope with growth	NFR-1-5	US-1, US-2	K-RT, K-BUS, C-REDIS, C-DB	ADR-001

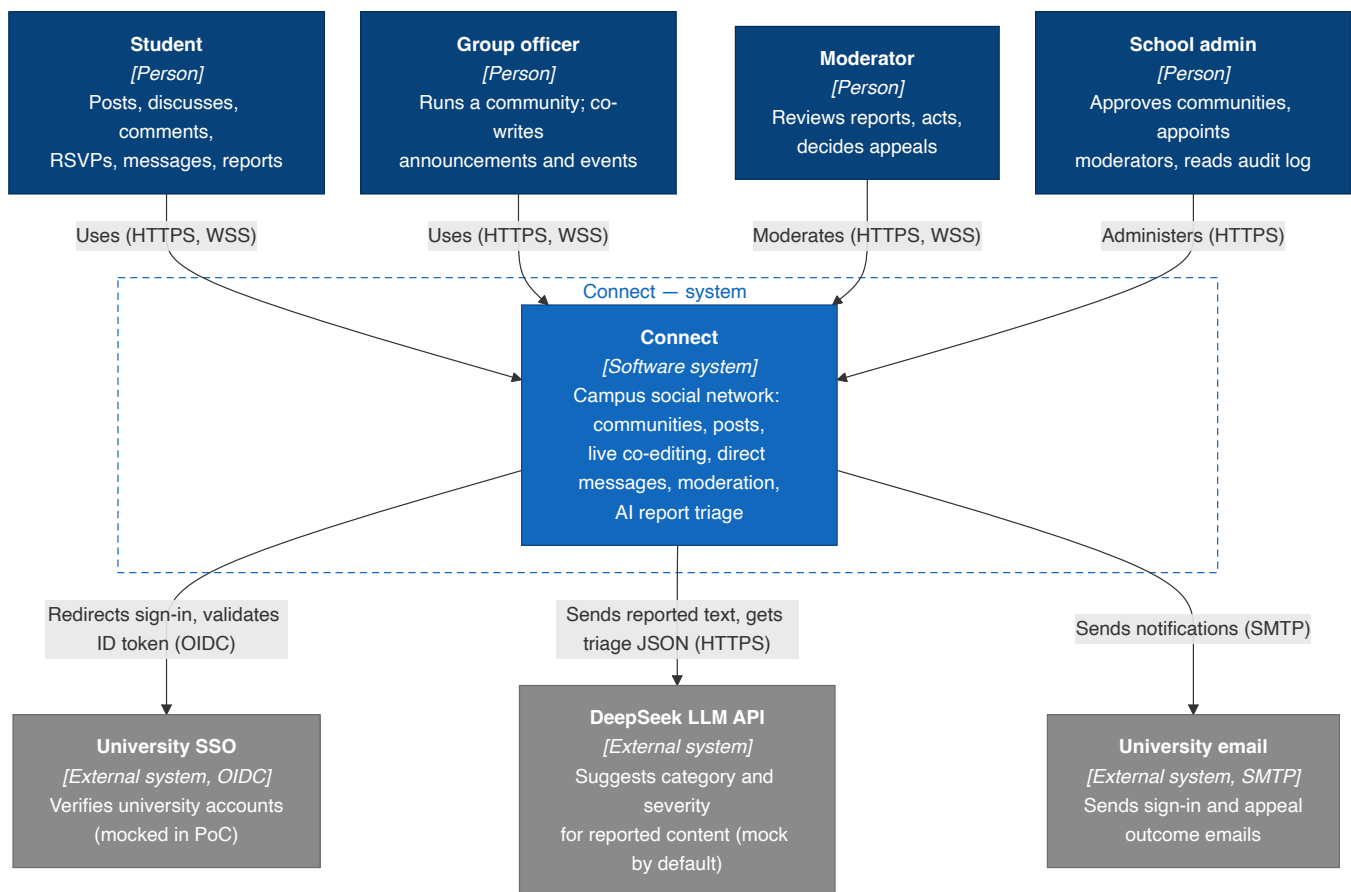
2 System Architecture

2.1 Architectural drivers (ranked)

Rank	Driver	Source	How it shaped the design
D1	Audience and authorization correctness	FR-AUD-*, FR-MOD-8, NFR-6	One Audience Policy component used by REST, feeds and WebSocket fan-out; 404 for hidden items; authorization only on the server (ADR-003).
D2	Real-time collaboration and live updates	FR-COL-*, FR-INT-4-6, NFR-2	WebSocket gateway in the API; Yjs CRDT for drafts (ADR-002); Redis pub/sub so every instance receives every event.
D3	Moderation accountability	FR-MOD-*, NFR-10	Moderation as its own module with an explicit state machine; insert-only audit log written in the same transaction as each action.
D4	Resilience to external and network failures	FR-AI-4, FR-COL-4, NFR-4/5	LLM calls moved to a worker behind an adapter (ADR-004); reconnect-and-resync for sockets; drafts saved as an append-only update log.
D5	Scalability with growing participation	NFR-1/3	Stateless API instances; shared state in PostgreSQL and Redis; cursor-based feeds.
D6	Team capacity and mandated technology	TC-1-4, SN-8	Modular monolith instead of microservices (ADR-001); mock services so each part can be built and tested alone.

2.2 System design (C4)

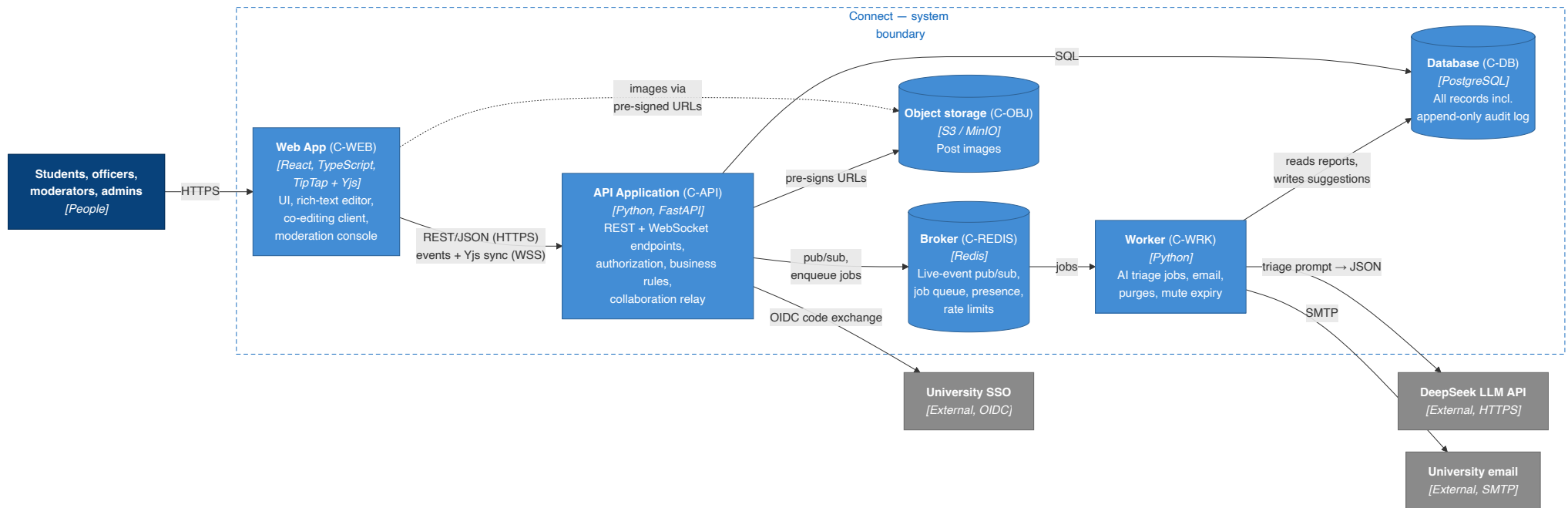
System context



System context diagram — Connect, its users and external systems (source: docs/diagrams/c4-context.mmd)

Connect is one software system used by four kinds of people through a browser. It depends on three external systems: **University SSO** proves that a person is a university member (FR-ID-1); the **DeepSeek LLM API** gives triage suggestions (FR-AI-1); **university email** delivers sign-in and appeal-result emails. All three are mocked in development so the system can be built and demonstrated without them.

Containers

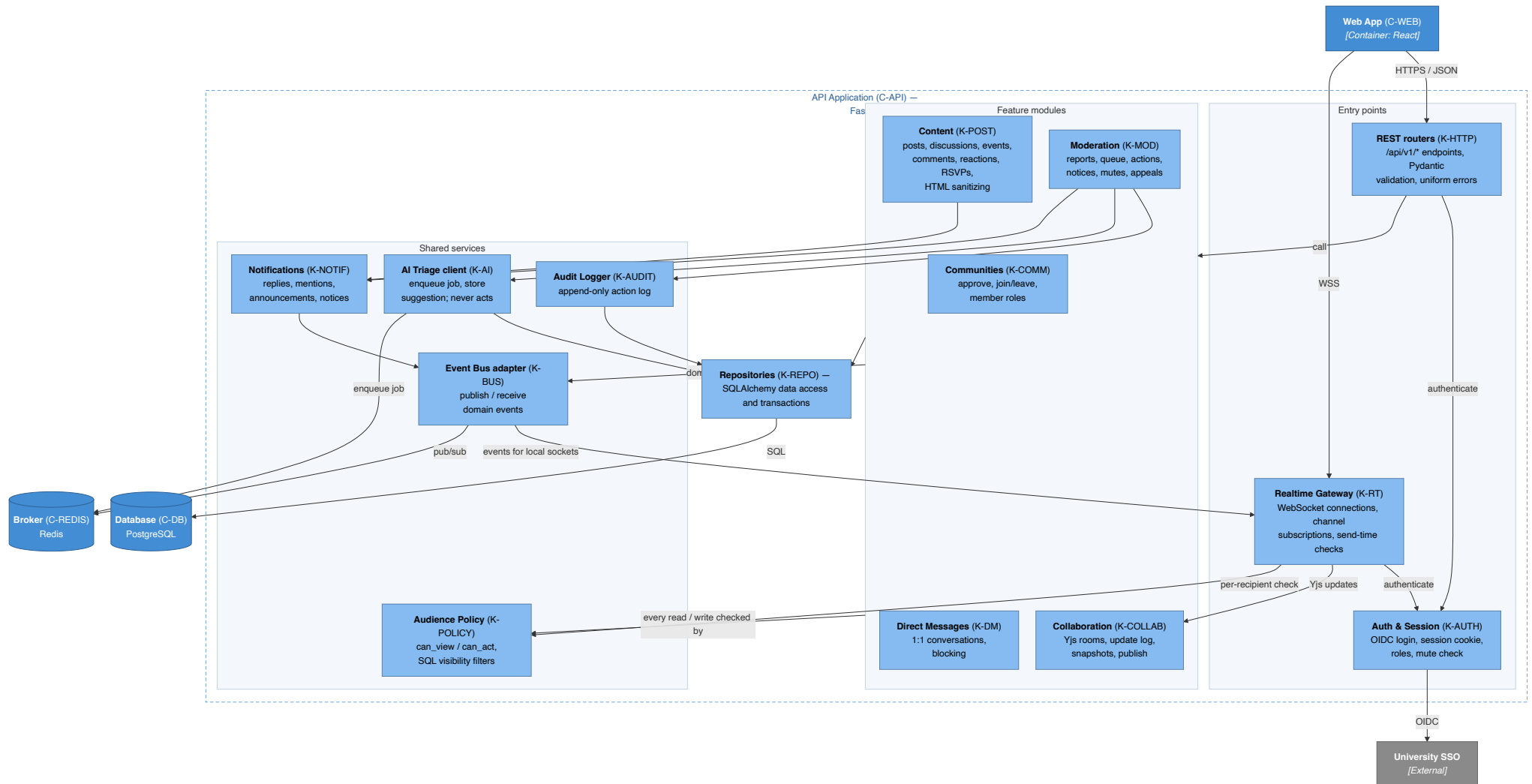


Container diagram — applications and data stores inside Connect (source: docs/diagrams/c4-container.mmd)

ID	Container	Responsibility	Technology
C-WEB	Web App	All user interfaces; rich-text editor; Yjs client for drafts; keeps one WebSocket open	React, TypeScript, Vite, TipTap, Yjs
C-API	API Application	REST API, WebSocket endpoints, authorization, business rules, collaboration relay	Python 3.12, FastAPI, Uvicorn, Pydantic, SQLAlchemy, pycrdt
C-WRK	Worker	AI triage jobs, emails, scheduled purges and mute expiry	Python, same codebase as C-API, Redis-backed queue
C-DB	Database	System of record including the audit log	PostgreSQL 16
C-REDIS	Cache / broker	Pub/sub of live events across API instances, job queue, presence, rate-limit counters	Redis 7
C-OBJ	Object storage	Images, uploaded with short-lived pre-signed URLs	S3-compatible (MinIO locally)

The browser talks only to C-API (and to C-OBJ through URLs that C-API signs). Only C-WRK holds the LLM key; only C-API holds the SSO client secret.

Components of the API Application



Component diagram — inside the API Application (source: docs/diagrams/c4-component-api.mmd)

We chose the API Application because it holds the rules that the requirements care most about. **REST routers** (K-HTTP) and the **Realtime Gateway** (K-RT) are the only entry points; both authenticate through **Auth & Session** (K-AUTH). Feature modules — **Communities** (K-COMM), **Content** (K-POST), **Direct Messages** (K-DM), **Collaboration** (K-COLLAB) and **Moderation** (K-MOD) — ask the **Audience Policy** (K-POLICY) before reading or changing anything. The Realtime Gateway asks K-POLICY again for each recipient before sending an event. Moderation writes to the **Audit Logger** (K-AUDIT) and asks the **AI Triage client** (K-AI) to enqueue a job. Domain events go through the **Event Bus adapter** (K-BUS) to Redis and come back to every instance's gateway. **Repositories** (K-REPO) are the only code that touches the database.

Main technology choices and how they fit Assignment 2

Choice	Why	Assignment 2 constraint
FastAPI + Uvicorn	Async I/O handles many idle WebSocket connections; Pydantic models generate the OpenAPI contract	TC-1, TC-2
React + TipTap + Yjs	TipTap gives accessible rich text and has official Yjs collaboration support	TC-3
PostgreSQL	Relational constraints express membership, visibility and "one appeal per action"; transactions make action + audit atomic	—
Redis	Lightweight pub/sub and queue; makes API instances stateless	TC-2, NFR-3
DeepSeek via LLMPProvider adapter	Low-cost API with JSON output mode; mock provider for tests; replaceable (Q3)	TC-4
Docker Compose for local development	Everyone runs the same PostgreSQL, Redis and MinIO versions	—

2.3 Behaviour and design decisions

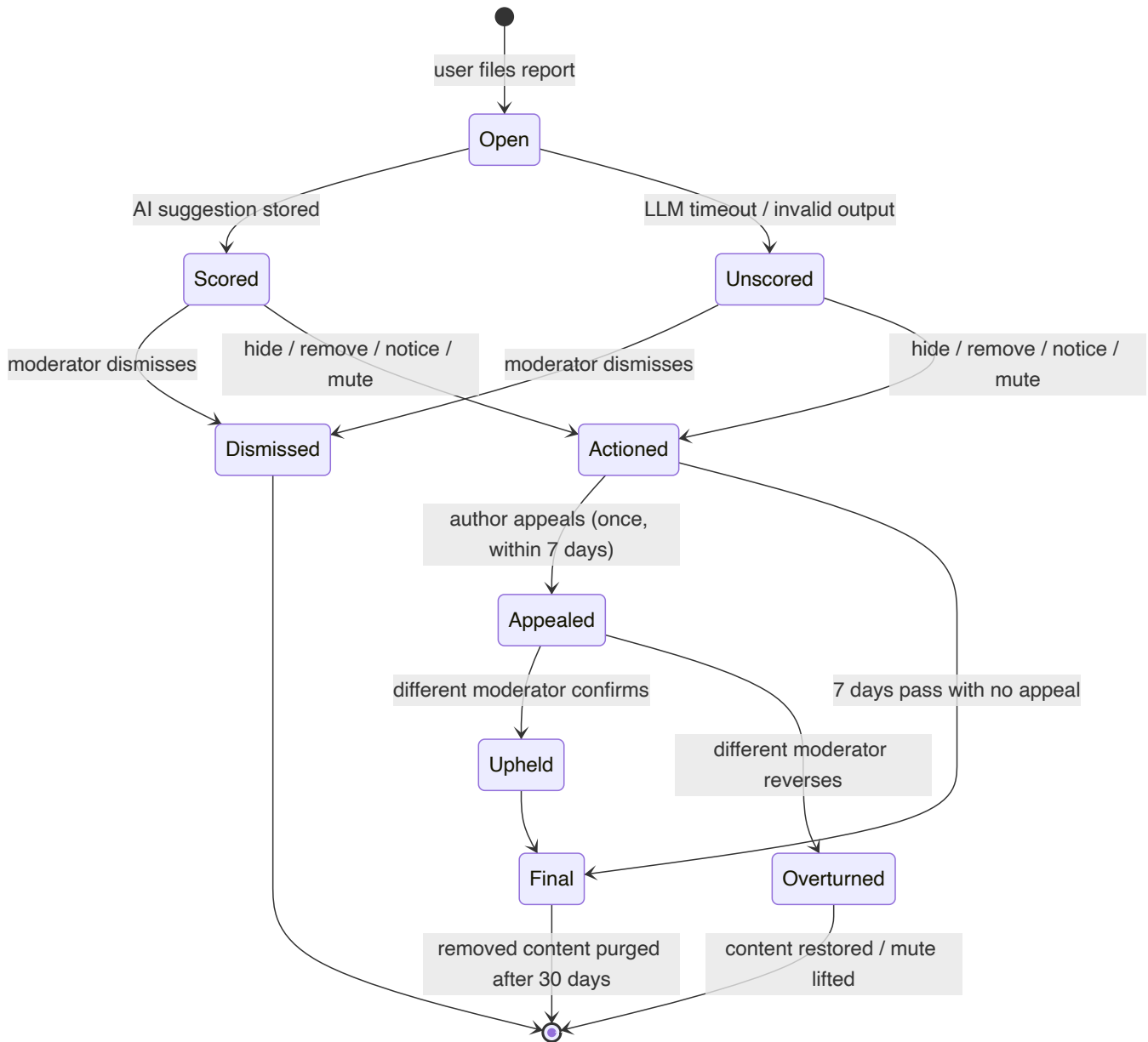
Audience and authorization (D1, ADR-003). Every request carries a session cookie; K-AUTH resolves the user, global role and active mutes. K-POLICY answers `can_view` / `can_act` and builds the SQL filter for list queries (visibility = campus, or community ∈ my memberships, or followers and I follow the author, minus blocked users, minus hidden/removed unless I am the author or a moderator). Single-item reads outside the audience return 404 `POST_NOT_FOUND`. Moderators pass all visibility checks except for DMs, which only participants can open (FR-MOD-8). Writes check role and mute status (403 `MUTED`, 403 `NOT_A_MEMBER`, 403 `NOT_AN_OFFICER`).

Publishing and collaboration (D2, ADR-002). Ordinary posts are created with a single REST call; edits use `If-Match: <version>` and fail with 409 `VERSION_CONFLICT` if someone else edited first. Announcements and events are written as Yjs drafts over `/ws/drafts/{id}`; the server relays, persists (append-only update log + periodic snapshots) and, on publish, converts and sanitizes the document inside one transaction with a version check, so only one publish wins (see sequence diagram).

Live updates (D2, D5). The Web App keeps one WebSocket on `/ws` and subscribes to channels it is allowed to see (`user:{me}`, `community:{id}`, `post:{id}`, `moderation`). When something changes, the owning module publishes a domain event to Redis; every API instance receives it and sends it to its local sockets after a per-recipient policy check. Events carry increasing IDs; after a reconnect the client sends `resume` with its last ID and gets missed events from a 5-minute Redis stream, or `resync_required` and reloads through REST. Clients ignore duplicate event IDs, so delivery is at-least-once and idempotent.

Direct messages. Conversations have exactly two members. Messages are stored, then delivered over `user:{id}` channels. Blocked or muted senders get 403. Reporting copies the selected messages into `REPORTED_MESSAGE`, so moderators never need conversation access.

Moderation and appeals (D3). Reports and actions follow the state machine below. Hiding and removing are status changes on the content row; the purge job deletes removed content after 30 days. Each action and its audit entry are written in one transaction. The appeal endpoint checks the 7-day window and uniqueness; a database constraint prevents the original moderator from reviewing the appeal.



Report and moderation state machine (source: docs/diagrams/moderation-states.mmd)

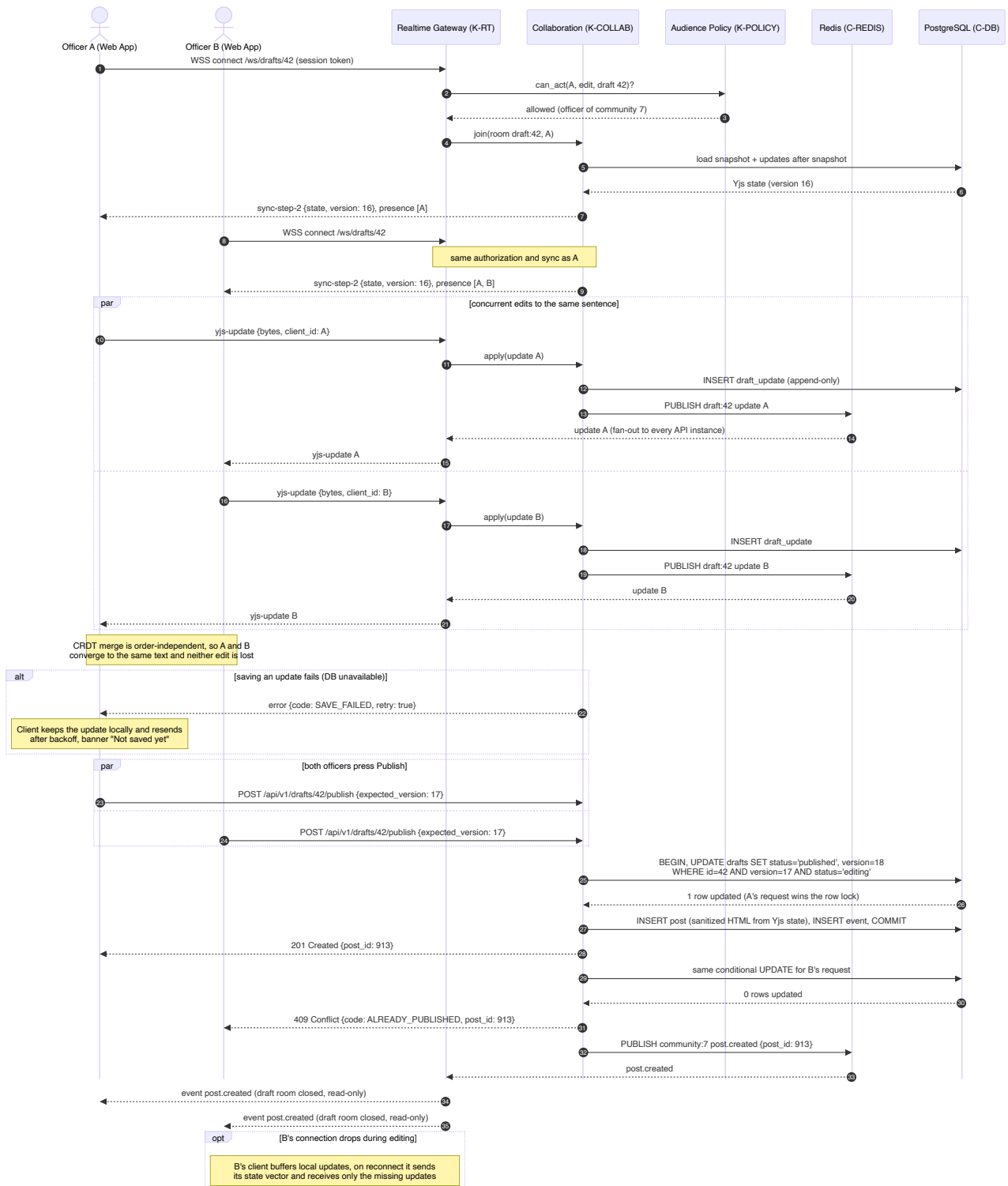
AI feature (D4, ADR-004). Filing a report commits it immediately (202). K-AI enqueues `triage(report_id)`; the worker loads the text, strips mentions and names, wraps the text in delimiters, calls the provider with a 10 s timeout and 2 retries, validates the JSON against the schema and stores an `AI_SUGGESTION`, then publishes `report.scored` to the moderation channel. Any failure marks the report `unscored`. The AI has no endpoint or permission that can change content.

Failures we planned for

Failure	Behaviour	Responsible
WebSocket disconnects	Client reconnects with exponential backoff (1 s → 30 s), resumes from last event ID or resyncs over REST; editor keeps local Yjs updates	C-WEB, K-RT
Redis unavailable	REST keeps working; live features show "live updates paused" and the client polls every 15 s; jobs wait in the queue	K-BUS, C-WEB
LLM timeout / invalid output / outage	Report marked <code>unscored</code> and queued within 5 s	C-WRK, K-AI
Database write fails during co-editing	Update not acknowledged; client retries; "Not saved yet" banner	K-COLLAB
Two officers publish at once	Conditional update; second gets 409 <code>ALREADY_PUBLISHED</code>	K-COLLAB
Two moderators act on the same report	Status check in the same transaction; second gets 409 <code>ALREADY_DECIDED</code>	K-MOD
Malicious HTML / prompt injection in reported text	Allowlist sanitizer on write; delimiter + enum-only output for the LLM	K-POST, C-WRK
API instance crash	Stateless; other instances keep serving; clients reconnect; at most 5 s of draft updates in flight	C-API

Trade-offs. A modular monolith is simpler to build and test than microservices but scales as one unit (ADR-001). CRDT editing gives the best experience but stores binary state and is harder to learn than locking (ADR-002). Soft deletion supports appeals and audit but keeps data longer, so we cap it at 30 days. Asynchronous AI keeps reporting robust but means suggestions arrive a few seconds later, and advisory-only AI means obvious cases still wait for a person (ADR-004). Returning 404 instead of 403 hides content existence but makes debugging slightly harder; logs record the real reason.

Sequence diagram: co-editing and publishing an event draft



Sequence diagram — two officers co-edit a draft and both press Publish (source: docs/diagrams/sequence-collab-publish.mmd)

Concurrent action. Steps in the first `par` block happen at the same time. Each update is appended to the log and broadcast through Redis, so it reaches officers connected to any API instance. Because Yjs merges are commutative, A and B converge to the same document whatever order updates arrive in. In the second `par` block both officers press Publish with `expected_version 17`; PostgreSQL's row lock lets one conditional update succeed and the other update zero rows, which K-COLLAB turns into **409 ALREADY_PUBLISHED**.

Failure. If an update cannot be saved, the server does not acknowledge it; the client keeps it and retries, so the edit is not lost. If B's connection drops, B keeps editing locally and on reconnect exchanges state vectors to receive only missing updates (FR-COL-4).

Who is responsible for what

Behaviour	Component
Sign-in, sessions, roles, mute check	K-AUTH
Visibility, membership, block and role decisions	K-POLICY
Communities, joining, roles	K-COMM
Posts, discussions, announcements, events, comments, reactions, RSVPs, sanitizing	K-POST
Direct messages	K-DM
Draft rooms, persistence, publish	K-COLLAB
Live delivery and subscriptions	K-RT + K-BUS
Reports, actions, notices, mutes, appeals	K-MOD
AI suggestions	K-AI (API side), C-WRK (LLM call)
Notifications	K-NOTIF
Audit log	K-AUDIT
Rendering, editor, offline banners, accessibility	C-WEB

Developing and testing components separately, then together

- **Unit:** K-POLICY with table-driven tests (role × visibility × relationship); moderation state machine; HTML sanitizer; LLM response validator.
- **Contract:** `shared/contracts/openapi.yaml` is the agreement. Backend contract tests fail if implemented routes or fields differ (already in the PoC). The frontend generates TypeScript types from it and develops against Mock Service Worker before the backend is ready.
- **Mocks:** mock SSO, `MockProvider` for the LLM, in-memory event bus, so each member can work without the others' parts.
- **Integration:** API tests with a real PostgreSQL and Redis in Docker; WebSocket tests with two clients (merge, publish race, reconnect).
- **End-to-end:** Playwright scripts for US-1 to US-6 in CI on every merge to `main`.
- **Non-functional:** k6 load test (NFR-1–3), axe-core (NFR-9), LLM evaluation set (NFR-12) at milestones.

2.4 Interfaces

Conventions. Base path `/api/v1`, JSON bodies, authentication by `HttpOnly` session cookie after SSO (the PoC uses the mock header `X-Demo-User`). Errors always have the form `{"error": {"code", "message", "details"?}}`. Common errors on every endpoint: 400 `VALIDATION_ERROR`, 401 `UNAUTHENTICATED`, 429 `RATE_LIMITED` (with `Retry-After`). Items the caller may not see return 404. The posts contract **C-POSTS** is fully specified in `shared/contracts/openapi.yaml` and implemented by the PoC.

REST endpoints

Method and path	Who may call	Request	Success	Specific errors
GET <code>/auth/login</code>	anyone	—	302 to SSO	—
GET <code>/auth/callback</code>	anyone	<code>code</code> , <code>state</code>	302 to app, sets cookie	400 <code>INVALID_STATE</code> , 403 <code>DOMAIN_NOT_ALLOWED</code>
POST <code>/auth/logout</code>	signed in	—	204	—
GET <code>/me</code>	signed in	—	200 user, roles, memberships	—
POST <code>/communities</code>	signed in	<code>name</code> , <code>description</code> , <code>join_policy</code>	202 pending approval	409 <code>NAME_TAKEN</code>
POST <code>/admin/communities/{id}/approve</code>	admin	<code>decision</code>	200	403 <code>NOT_ADMIN</code>
POST <code>/communities/{id}/join</code>	signed in	—	200 joined / 202 requested	409 <code>ALREADY_MEMBER</code>
DELETE <code>/communities/{id}/members/me</code>	member	—	204	—
PATCH <code>/communities/{id}/members/{user}</code>	owner	<code>role</code>	200	403 <code>NOT_OWNER</code>
POST <code>/posts</code> (C-POSTS)	signed in; member if community	<code>type</code> , <code>community_id</code> , <code>visibility</code> , <code>title</code> , <code>body_html</code>	201 Post + <code>Location</code>	403 <code>NOT_A_MEMBER</code> , 403 <code>MUTED</code> , 404 <code>COMMUNITY_NOT_FOUND</code>
GET <code>/posts</code> (C-POSTS)	signed in	<code>community_id?</code> , <code>limit</code>	200 visible posts	—
GET <code>/posts/{id}</code> (C-POSTS)	in audience	—	200 Post	404 <code>POST_NOT_FOUND</code>

Method and path	Who may call	Request	Success	Specific errors
PATCH /posts/{id}	author	fields + If-Match: version	200	403, 409 VERSION_CONFLICT
DELETE /posts/{id}	author	—	204	403
POST /posts/{id}/comments	in audience	body, parent_id?	201	403 COMMENTS_DISABLED, 423 LOCKED
PUT / DELETE /posts/{id}/reactions/{kind}	in audience	—	204	—
PUT /events/{id}/rsvp	in audience	status	200 going / waitlisted	410 RSVP_CLOSED
POST /communities/{id}/drafts	officer	kind, collaborator_ids	201 draft	403 NOT_AN_OFFICER
POST /drafts/{id}/publish	officer collaborator	expected_version, visibility, event fields	201 post	409 ALREADY_PUBLISHED, 409 VERSION_CONFLICT
POST /conversations	signed in	user_id	201 / 200 existing	403 BLOCKED
GET / POST /conversations/{id}/messages	participant	body (POST)	200 / 201	404 not participant, 403 MUTED, 403 BLOCKED
POST /reports	in audience, not author	target_type, target_id, reason, note, message_ids?	202 report	409 DUPLICATE_REPORT
GET /moderation/queue	moderator	status?	200 reports + AI suggestions	403 NOT_MODERATOR
POST /moderation/reports/{id}/actions	moderator	action, reason, mute_duration?, ai_agreed	201 action	409 ALREADY_DECIDED
POST /appeals	affected user	action_id, statement	201 appeal	409 ALREADY_APPEALED, 410 APPEAL_WINDOW_CLOSED
POST /moderation/appeals/{id}/decision	moderator ≠ original	upheld / overturned, reason	200	403 SAME_MODERATOR
GET /admin/audit-log	admin	from, to, format=csv	200 CSV	403 NOT_ADMIN
GET /notifications	signed in	cursor	200	—

Example (C-POSTS)

```
POST /api/v1/posts
X-Demo-User: aisha          # PoC only; real system uses the session cookie
Content-Type: application/json

{"type": "post", "community_id": 1, "visibility": "community",
 "title": null, "body_html": "<p>Robotics meetup moved to <b>Room 204</b>.</p>"}
```

```
HTTP/1.1 201 Created
Location: /api/v1/posts/2

{"id": 2, "type": "post", "author": {"id": "aisha", "display_name": "Aisha (student)"},
 "community_id": 1, "visibility": "community", "title": null,
 "body_html": "<p>Robotics meetup moved to <b>Room 204</b>.</p>",
 "status": "visible", "version": 1, "created_at": "2026-10-06T07:32:46Z"}
```

WebSocket /ws (JSON frames, cookie authentication)

Direction	Frame	Notes
client → server	{"type": "subscribe", "channel": "community:12"}	Rejected with {"type": "error", "code": "FORBIDDEN", "channel": "...} if not allowed
client → server	{"type": "resume", "last_event_id": "1696..-3"}	Server replays or sends {"type": "resync_required"}
server → client	{"type": "event", "id": "...", "channel": "community:12", "event": "post.created", "occurred_at": "...", "data": {Post}}	Events: post.created/updated/hidden, comment.created, reaction.changed, rsvp.changed, message.created, notification.created, report.created, report.scored
both	ping / pong every 25 s	Idle sockets closed after 60 s

/ws/drafts/{id} carries the binary Yjs sync protocol (sync step 1/2, update, awareness). Close codes: 4401 unauthenticated, 4403 not a collaborator, 4409 draft already published.

LLM adapter (worker → DeepSeek)

```

TriageRequest { text: string (≤ 6000 chars, names stripped), reason: ReportReason,
               categories: [ReportReason], policy_version: string }
TriageResult  { category: ReportReason, severity: "low"|"medium"|"high",
               rationale: string (≤ 300 chars) }
Failure       timeout 10 s, 2 retries with backoff; invalid JSON or enum → report "unscored"

```

2.5 Code and repository structure

```

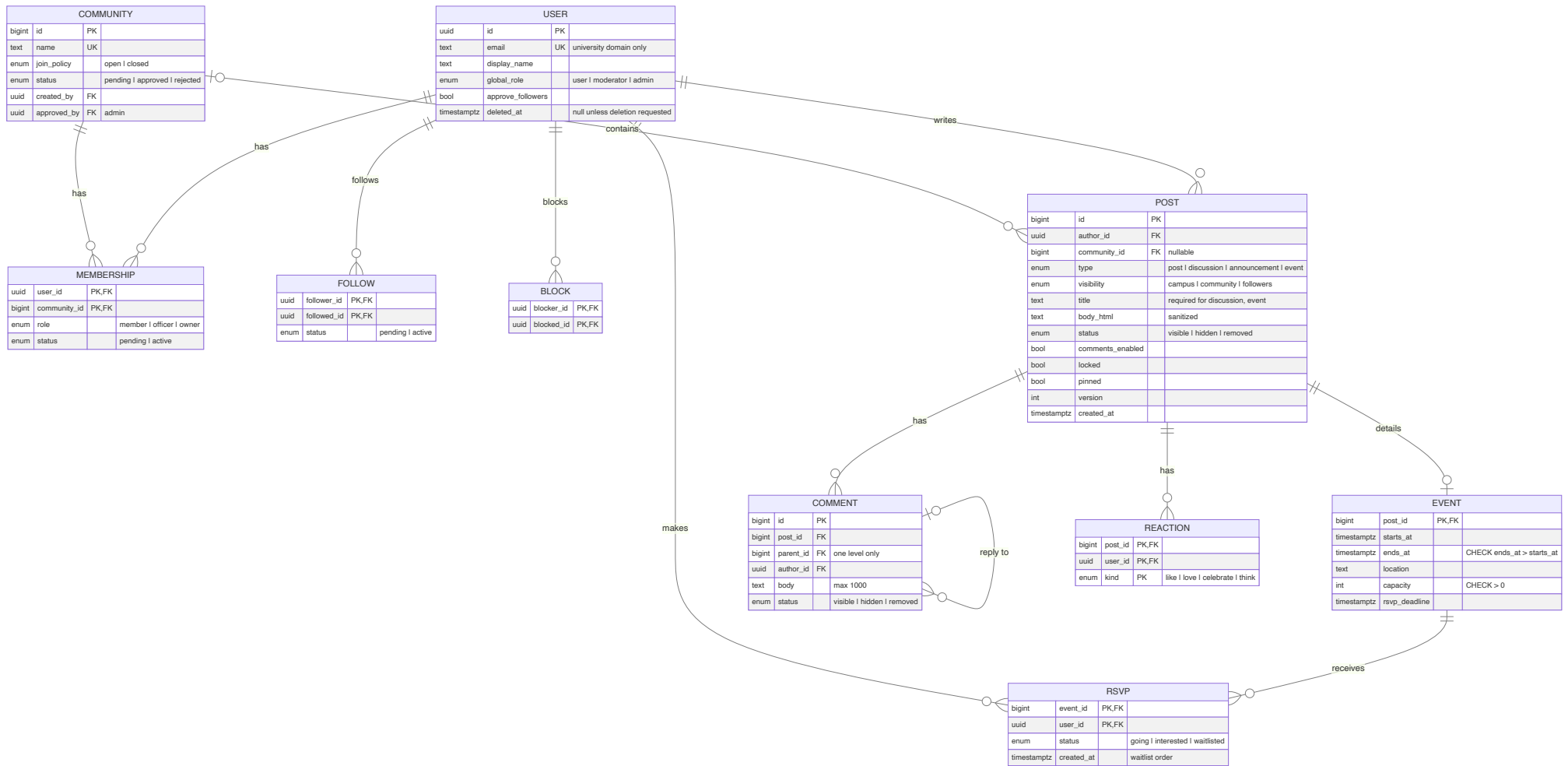
AmazingNetwork/
├── backend/                                # C-API and C-WRK (one Python codebase)      owner: A (Jiadong)
│   ├── app/
│   │   ├── api/                          # K-HTTP routers                        A
│   │   ├── auth/                        # K-AUTH                                A
│   │   ├── policy.py                    # K-POLICY (reviewed by A + D)          A
│   │   ├── communities/ content/ dm/    A
│   │   ├── realtime/ collab/ bus/      # K-RT, K-COLLAB, K-BUS                B (Nura)
│   │   ├── moderation/ audit/ ai/     # K-MOD, K-AUDIT, K-AI                D (Zayed)
│   │   ├── repositories/ models/ schemas/ A
│   │   └── worker/                     # C-WRK jobs, LLM providers                D
│   ├── migrations/                     # Alembic                                A
│   └── tests/ unit/ integration/ contract/ D
├── frontend/                            # C-WEB (React + TS; PoC is plain HTML/JS)  C (Fatima)
│   └── src/ features/{feed,communities,drafts,messages,moderation}/ api/
├── shared/contracts/                   # openapi.yaml, ws-events.schema.json → TS types  A + C
├── e2e/                               # Playwright tests for US-1..US-6        D
├── docs/ report/ diagrams/ adr/        D (all write)
├── infra/                             # docker-compose.yml, Postgres/Redis/MinIO config B
├── .github/                           # CI workflow, CODEOWNERS, PR template      B
└── .env.example .gitignore README.md DECISIONS.md

```

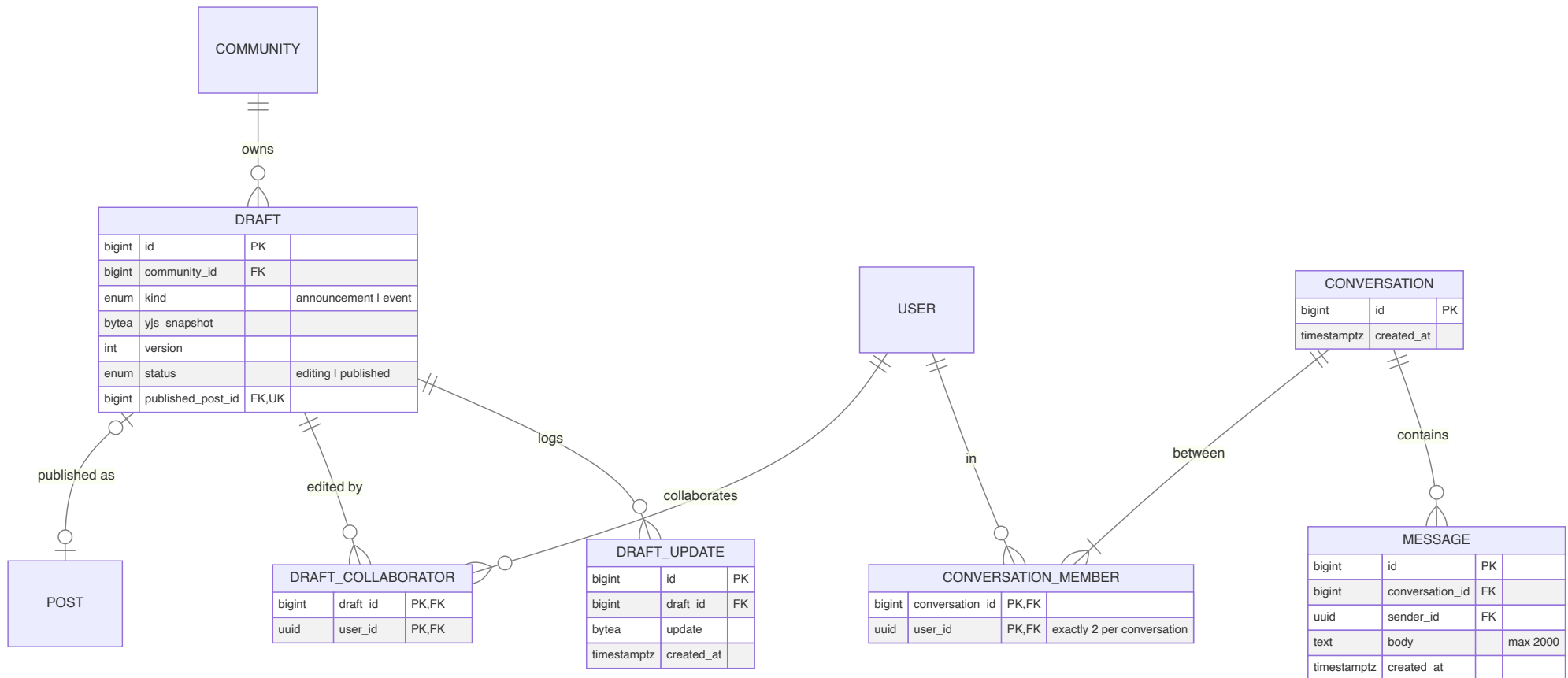
Top-level folders follow C4 containers; backend packages follow C4 components, so a reviewer can go from a diagram box to its code. `CODEOWNERS` assigns each folder to its owner and requires both a backend and a frontend reviewer for `shared/contracts/`.

Keeping secrets out. Real values live only in `.env` (git-ignored); `.env.example` holds placeholders. The backend reads settings from environment variables. CI uses GitHub Actions secrets and runs `gitleaks` on every push. The browser bundle receives only public configuration (API base URL): the DeepSeek key is read only by the worker and the SSO client secret only by the API, so neither can appear in frontend code. Any leaked key is rotated immediately and the leak noted in `DECISIONS.md`.

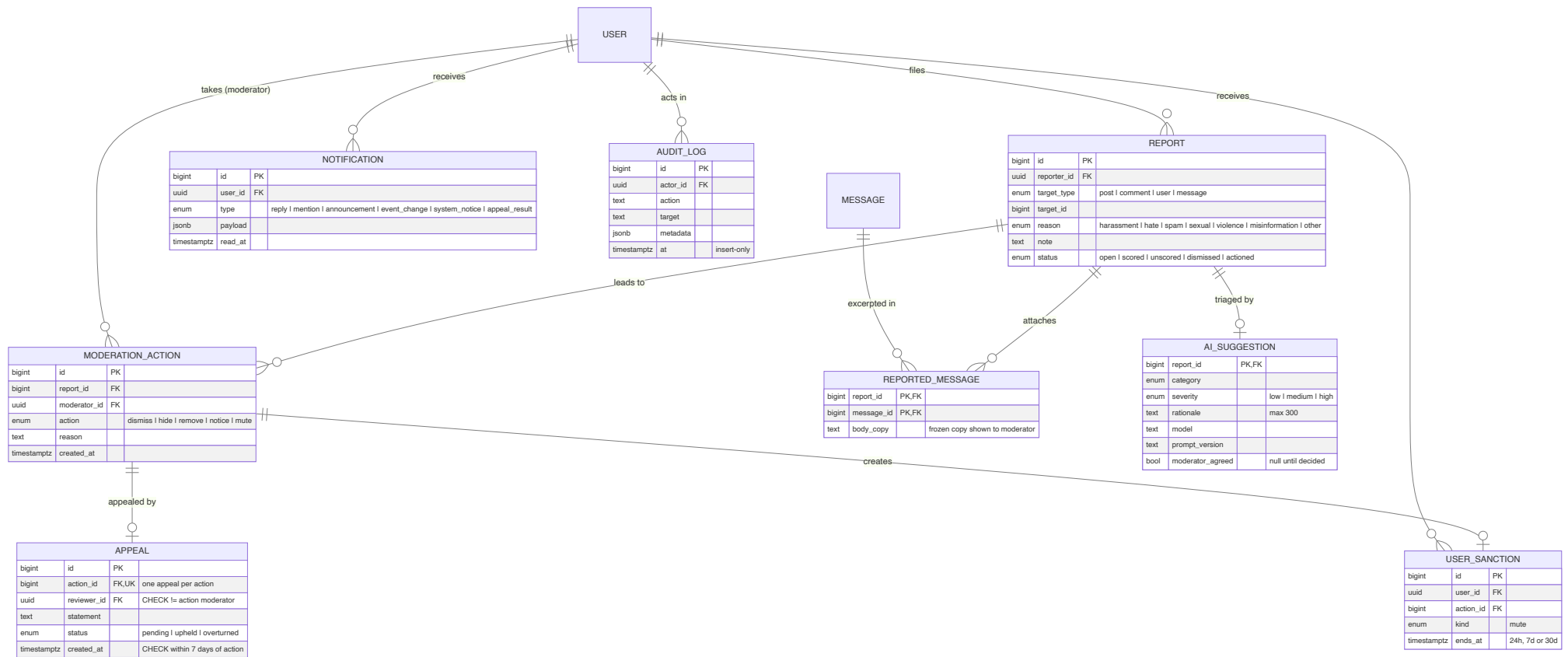
2.6 Data model



Entity relationship diagram, part 1 — people, communities and published content (source: docs/diagrams/erd-1-people-content.mmd)



Entity relationship diagram, part 2 — collaborative drafts and direct messages (source: docs/diagrams/erd-2-drafts-messages.mmd)



Entity relationship diagram, part 3 — moderation, AI triage, notifications and audit (source: docs/diagrams/erd-3-moderation.mmd)

The ERD is split into three parts so it stays readable; entities shown without attributes (e.g. USER in parts 2 and 3) are defined in another part.

Entities. USER, COMMUNITY and MEMBERSHIP model identity and groups (role per community). FOLLOW and BLOCK hold relationships used by the Audience Policy. POST holds all published types with EVENT as a 1:1 extension and RSVP, COMMENT and REACTION hanging off it. DRAFT, DRAFT_COLLABORATOR and DRAFT_UPDATE support co-editing. CONVERSATION, CONVERSATION_MEMBER and MESSAGE hold DMs. REPORT, REPORTED_MESSAGE, AI_SUGGESTION, MODERATION_ACTION, APPEAL and USER_SANCTION model moderation; NOTIFICATION and AUDIT_LOG support delivery and accountability.

Constraints and the requirements they enforce

Constraint	Enforces
USER.email unique, domain checked at sign-in	FR-ID-1
MEMBERSHIP primary key (user, community); role ∈ member/officer/owner	FR-ID-3, FR-COM-3
CHECK visibility = 'community' ⇒ community_id IS NOT NULL; CHECK visibility = 'followers' ⇒ community_id IS NULL	FR-AUD-1
CHECK type IN ('announcement', 'event') ⇒ author is officer (enforced in K-POLICY)	§0.5 table
POST.version with conditional update	FR-PUB-5, FR-COL-5
DRAFT.published_post_id unique	FR-COL-5 — a draft publishes once
EVENT.capacity > 0, ends_at > starts_at; RSVP primary key (event, user)	FR-PUB-4
REACTION primary key (post, user, kind)	FR-INT-2
CONVERSATION_MEMBER: exactly 2 rows per conversation	FR-INT-6
REPORT unique (reporter, target_type, target_id)	FR-MOD-1 — one report per item per user
APPEAL.action_id unique; CHECK reviewer ≠ action's moderator; created within 7 days	FR-MOD-5
USER_SANCTION.ends_at from allowed durations	P4
AUDIT_LOG insert-only (application DB role has no UPDATE/DELETE)	FR-MOD-6, NFR-10
Content status ∈ visible/hidden/removed; purge job deletes removed after 30 days	P1, NFR-7

2.7 Architecture decision records

The four decisions with the greatest effect on the system, from four different parts of it. Full text is kept in `docs/adr/`.

ADR-001: Modular monolith plus one background worker

Status: Accepted · **Area:** overall structure / deployment · **Drivers:** D1, D5, D6

Context. Connect needs REST, WebSockets, background AI calls and scheduled purges. The team has four students, about six weeks for Assignment 2, and must use FastAPI. Audience rules (FR-AUD-*) must be enforced identically by every feature, and several operations (publishing a draft, taking a moderation action and writing the audit entry) must be atomic.

Options considered.

1. *Microservices* (separate posts, messaging, moderation, AI services). Independent scaling, but needs service-to-service authentication, distributed transactions and much more deployment work; audience rules would be duplicated or require network calls.
2. *Single FastAPI process doing everything*, including LLM calls inside the request. Simplest, but a slow or failing LLM would block request workers and scheduled jobs would have no home.
3. *Modular monolith (one FastAPI app with strict internal modules) plus one worker process* sharing PostgreSQL and Redis.

Decision. Option 3. Modules map to C4 components (K-POST, K-MOD, ...). Modules talk through Python interfaces, never through each other's tables. Slow or external work (LLM triage, email, purges) goes to the worker through a Redis queue.

Consequences.

- One deployable API, one database transaction per use case (e.g. moderation action + audit entry commit together).
- The API is stateless (sessions in DB, live events through Redis pub/sub), so it scales horizontally by running more instances (NFR-3).
- Module boundaries are enforced only by convention and review (CODEOWNERS, import rules checked in CI), not by the network.
- If one module later needs independent scaling (e.g. realtime), it can be split out along its existing interface.

ADR-002: Yjs CRDT for real-time co-editing of drafts

Status: Accepted · **Area:** publishing and collaboration · **Drivers:** D2, D4

Context. Officers co-write announcements and events (FR-COL-1..5). Edits must reach other editors within 500 ms (NFR-2), simultaneous edits must not lose text (FR-COL-3), and a dropped connection must not lose work (FR-COL-4). Assignment 2 requires a

React rich-text editor and WebSockets.

Options considered.

1. *Last-writer-wins with a version number.* Trivial, but the second save overwrites or is rejected, so real simultaneous editing is impossible.
2. *Paragraph locking* (one editor per paragraph). Simple to reason about, but blocks people, needs lock timeouts, and offline edits cannot be merged.
3. *Operational transformation (OT).* Proven (Google Docs) but the server must transform every operation; no maintained Python implementation fits our stack.
4. *CRDT with Yjs.* The TipTap editor has a Yjs collaboration extension; the Python `pycrdt` / `pycrdt-websocket` libraries speak the same protocol, so the FastAPI server can relay and persist updates.

Decision. Option 4. Each draft is a Yjs document in room `draft:{id}`. The server checks authorization on connect, relays updates through Redis to all API instances, appends each update to `draft_update` and writes a snapshot every 100 updates. Publishing converts the Yjs state to HTML, sanitizes it and uses a conditional `UPDATE ... WHERE version = expected_version` so only one publish succeeds.

Consequences.

- Concurrent edits converge automatically; offline edits merge on reconnect.
- Drafts are stored as binary state, so the server cannot validate content until publish; validation and sanitizing happen at publish time.
- Team must learn Yjs; risk R1 mitigated by a week-1 spike. Fallback: option 2 behind the same WebSocket endpoint.
- Only drafts use CRDTs; published posts use ordinary versioned rows with `If-Match` (409 on conflict).

ADR-003: One server-side Audience Policy component for every read path

Status: Accepted · **Area:** audience control and authorization · **Drivers:** D1

Context. A post must never reach someone outside its audience through *any* path: single fetch, feed, search, comments, notifications, mentions or WebSocket events (FR-AUD-2). Rules combine visibility, membership, follows, blocks, roles, moderation status and the DM exception for moderators. Leaks are the highest-impact failure for students (SN-2) and the university (SN-6).

Options considered.

1. *Checks written inside each endpoint.* Fast to start, but rules drift between endpoints and WebSocket fan-out is easy to forget.
2. *Filter in the frontend.* Unacceptable: data has already left the server.
3. *Central policy component (K-POLICY)* offering `can_view(user, item)`, `can_act(user, action, item)` and a SQL filter builder used by list queries; the Realtime Gateway calls it again for each recipient before sending an event.
4. *Database row-level security.* Strong guarantee, but harder to test and debug for the team and awkward for the WebSocket path.

Decision. Option 3. All endpoints and the WebSocket gateway depend on K-POLICY. Content outside the caller's audience returns **404 POST_NOT_FOUND** (identical to a missing post) so its existence is not revealed. The proof of concept already implements this (`backend/app/policy.py`).

Consequences.

- One place to review and test: table-driven unit tests cover every role × visibility × relationship combination (NFR-11).
- Small extra cost per event (a policy check per recipient); acceptable at campus scale, and cached membership sets keep it cheap.
- New endpoints must use the policy; a CI test lists all routes and fails if a content route lacks the policy dependency.
- Visibility changes take effect immediately because nothing caches permission decisions beyond one request.

ADR-004: AI report triage is advisory, asynchronous and behind a provider adapter

Status: Accepted · **Area:** AI feature and moderation · **Drivers:** D3, D4

Context. Moderators are few and report volume grows with participation (SN-5). An LLM can suggest a category and severity so urgent reports are seen first (FR-AI-*). But LLM output can be wrong, can be manipulated by text inside the reported content (prompt injection), and sending content to an external provider (DeepSeek) raises privacy questions (SN-6, Q3). Assignment 2 requires an LLM integration; paid calls are not required for Assignment 1.

Options considered.

1. *Automatic moderation:* the AI hides high-severity content immediately. Fast, but wrong decisions silence students without human review, and an attacker could game it.
2. *Synchronous call while filing the report.* Simple, but the reporter waits up to 10 s and an outage blocks reporting.
3. *Advisory, asynchronous triage:* the report is saved first, a worker job calls the LLM, the result is validated against a JSON schema and stored as a suggestion; moderators always decide.

Decision. Option 3, with the LLM behind an `LLMProvider` interface that has a `MockProvider` (default, used in tests and demos) and a `DeepSeekProvider` (JSON output mode, 10 s timeout, 2 retries with backoff). The prompt contains only the reported text, the report

reason and the policy categories; no names, user IDs or conversation history. Reported text is wrapped in delimiters and treated as data. Output fields must be enum values or the report is marked *unscored*.

Consequences.

- No report is ever lost or delayed by the AI (report appears in the queue within 5 s either way).
- Obviously harmful content still waits for a human; we accept this to protect fairness and appeals.
- Moderator agree/disagree is recorded, giving an ongoing accuracy measure (NFR-12) and a basis to change prompts (`prompt_version` stored per suggestion).
- Switching to another provider or a local model touches one class only, which keeps the option open if the university rejects external processing (Q3).

3 Project Management and Team Collaboration

3.1 Team structure

Member	Responsibilities	Code areas maintained	Backup for
Jiadong Zhang (A)	Backend core, data model, Audience Policy, API contract	backend/app/{api,auth,policy.py,communities,content,dm,repositories,models,schemas}, migrations/, shared/contracts/	Moderation (D)
Nura (B)	Real-time, collaboration, infrastructure, CI	backend/app/{realtime,collab,bus}, infra/, .github/	Backend core (A)
Fatima (C)	Frontend, editor, accessibility, usability tests	frontend/, shared/contracts/ (co-owner)	Real-time (B)
Zayed (D)	Moderation, AI triage, test strategy, documentation	backend/app/{moderation,audit,ai,worker}, backend/tests/, e2e/, docs/	Frontend (C)

Coordinating changes. The API contract is changed first, in its own pull request reviewed by A and C, before code that depends on it. Cross-area changes are split into PRs per owner and linked to one issue. A 15-minute stand-up happens three times a week (two online, one in person) and a weekly integration session merges and runs the end-to-end tests together.

Sharing knowledge. Each area has a backup owner who reviews its PRs. Every week one member gives a 20-minute walkthrough of their area, rotating, so that all four can explain the whole system in the oral exam. ADRs and `DECISIONS.md` record why things are the way they are.

3.2 Development workflow

- **Branches:** GitHub Flow. `main` is protected (no direct pushes, CI must pass). Branches are named `feat/<issue>-<slug>`, `fix/...`, `docs/...`.
- **Pull requests:** linked to an issue; template asks for affected requirement IDs, tests, contract and ADR changes and AI-tool use. One approving review from the area owner or backup; two (A + C) for `shared/contracts/` and K-POLICY. Squash-merge with a Conventional Commit title.
- **Issues and tracking:** GitHub Projects board (Backlog → Sprint → In progress → In review → Done). Labels: area (backend, frontend, realtime, moderation, ai, docs), priority (must, should, could), type (feature, bug, req-change, tech-debt). Each sprint is a GitHub milestone.
- **Decisions and requirement changes:** architectural decisions as ADRs in `docs/adr/` (reviewed in PRs). A requirement change gets a `req-change` issue; its PR updates the report, the traceability matrix and `DECISIONS.md` together, so the documents never disagree.
- **AI development tools:** allowed for drafting code, tests and text. The author must read, run and be able to explain every line before opening a PR, and states in the PR where AI was used. AI-generated code gets the same review and test requirements as any other code. No secrets, real user data or private course material are pasted into AI tools. Because the grade is an oral exam without AI, the weekly walkthroughs double as practice in explaining AI-assisted work.

3.3 Development approach

We use **light Scrum with one-week sprints**. Short sprints suit a six-week project with four people and two risky technologies (CRDT editing and LLM integration): every week ends with something we can demonstrate and a chance to change direction.

- **Iterations:** Monday planning (30 min) picks issues from the backlog; Friday review demo (20 min) and retrospective (10 min).
- **Prioritization:** MoSCoW linked to requirement IDs. *Must* = everything needed for US-1–US-6; *should* = pinning, waitlist, community mute; *could* = images, follower approval. Risky items go first (spikes in week 1).
- **Feedback:** Friday demos to classmates or the TA; usability test with five students at M3 and M5 (NFR-8).
- **Testing, documentation and technical work** are part of the Definition of Done: tests written, CI green, docs/ADR updated, accessibility checked for UI changes. About 15 % of each sprint is reserved for technical debt and bug fixing.

3.4 Risks

ID	Risk	Likelihood	Impact	Mitigation / response
R1	Yjs + FastAPI integration is harder than expected	Medium	High	Spike in week 1; if not working by end of week 2, fall back to paragraph locking on the same endpoint (ADR-002)
R2	An audience rule leaks content	Medium	High	Single K-POLICY, table-driven tests, negative E2E tests for every story, two reviewers for policy changes

ID	Risk	Likelihood	Impact	Mitigation / response
R3	DeepSeek unavailable, rate-limited or not approved by the university (Q3)	Medium	Medium	Mock provider by default; adapter allows a local model; triage is optional for the rest of the system
R4	LLM gives wrong or manipulated suggestions	Medium	Medium	Advisory only; enum-only output; evaluation set; agreement tracking
R5	Scope creep (shorts, groups chats, mobile app)	High	Medium	Scope table in §0.3; changes only through req-change issues; scope freeze after M2
R6	A member is ill or unavailable	Medium	High	Backup owner per area; weekly walkthroughs; small PRs
R7	Late integration between frontend and backend	Medium	Medium	Contract first, generated types, mock server, weekly integration session
R8	A member cannot explain AI-generated code in the oral exam	Medium	High	PR rule "explain every line"; rotating walkthroughs; pair review
R9	Secrets committed to the repository	Low	High	.gitignore, .env.example, gitleaks in CI, key rotation

3.5 Timeline for Assignment 2

Assumes six one-week sprints; dates will be fixed when Assignment 2 is released.

Milestone	End of	What will work (demonstrable / verifiable)
M1 Foundations	Week 1	docker compose up starts all containers; mock SSO sign-in; create and join a community; CI green. Yjs spike shows two browsers editing one document through FastAPI.
M2 Content and audience	Week 2	Create, edit and delete posts and discussions with rich text and all three visibilities; comments and reactions; automated tests prove a non-member gets 404 and never sees the post in the feed. Scope freeze.
M3 Live	Week 3	A new post, comment or DM appears in another browser within 1 s without refresh; reconnect after network loss resumes events; first usability test.
M4 Collaboration	Week 4	Two officers co-edit an event draft live; concurrent edits merge; offline edits sync; simultaneous publish gives exactly one event and one 409; RSVP and waitlist work.
M5 Moderation and AI	Week 5	Reports (including DM excerpts) appear in the queue with DeepSeek or mock suggestions; LLM outage gives "unscored"; hide, remove, notice, mute; appeal to a different moderator; audit export.
M6 Quality and release	Week 6	k6 shows NFR-1/2 targets met; axe shows 0 serious issues; LLM evaluation $\geq 80\%$; Playwright passes for US-1 to US-6; documentation updated; demo rehearsed by all four members.

4 Proof of Concept

What it does. A FastAPI backend and a plain HTML/JavaScript page communicate through contract **C-POSTS** (`shared/contracts/openapi.yaml`): create a post (`POST /api/v1/posts`), fetch it (`GET /api/v1/posts/{id}`) and list the caller's feed (`GET /api/v1/posts`). The page shows the exact request and response so it can be compared with the contract. Four fictional users demonstrate the audience rules: a non-member asking for a community post receives the same 404 as for a missing post.

What matches the planned architecture. Folder layout (`backend/`, `frontend/`, `shared/contracts/`); routers, schemas, Audience Policy (`app/policy.py`) and repository (`app/store.py`) as separate modules mirroring K-HTTP, K-POLICY and K-REPO; the uniform error format; server-side HTML allowlist sanitizing (FR-PUB-6); settings from environment variables.

What we simplified

Planned	In the PoC	Why acceptable
React + TipTap editor	Plain HTML form; rich text typed as HTML	Brief allows a minimal browser interface
SSO + session cookie	X-Demo-User header naming a fictional user	External systems may be mocked
PostgreSQL	In-memory repository with the same methods	Only the interaction is assessed
WebSockets, Redis, worker, LLM	Not included	Not required for the PoC
Rate limits, audit log, images	Not included	Not part of the chosen interaction

Evidence. 29 automated tests pass (`pytest in backend/`): API tests for success and every error code, table-driven tests for the Audience Policy, and a contract test that compares the running application with `openapi.yaml` (paths, request fields, response fields, enums and error shape). Setup and run instructions are in the root `README.md`; the three-minute demonstration follows `docs/demo-script.md`.

Use of AI tools. AI tools were used to help draft parts of this report, the diagrams and the PoC code. All four members reviewed, ran and edited the result and can explain every part of it; the team takes responsibility for the submission.